ideis · Uniwersytet DSW

Architektura Aplikacji w Pythonie

[Lab 3] Testowanie aplikacji

Michał Madejski

Laboratoria · 11 kwietnia 2026

Co umiesz po tych zajęciach?

Trzy konkretne umiejętności, które zabierzesz do domu:

01

Napiszesz test jednostkowy

...zarówno w unittest, jak i w pytest. Zrozumiesz różnicę i wybierzesz właściwe narzędzie.

02

Użyjesz setUp i fixtures

Zrozumiesz, jak izolować testy, żeby jeden nie psuł drugiego — fundament profesjonalnego testowania.

03

Sparametryzujesz testy

Jeden test, wiele zestawów danych. Mniej kodu, lepsze pokrycie, szybsze debugowanie.

Na końcu zajęć wracamy do tej listy — sprawdzimy, czy wszystko zaskoczyło.

ideis

Agenda · 90 minut

0:00 – 0:15	**Wstęp**	po co testy · assert · typy testów
0:15 – 0:45	**unittest**	klasy testowe · setUp · DEMO · błędy
0:45 – 0:50	**Przerwa**	5 min na pytania
0:50 – 1:10	**pytest**	fixtures · parametryzacja · marks
1:10 – 1:30	**Zadania**	klasa Product · samodzielnie

ideis

Dlaczego testujemy?

Knight Capital · 1 sierpnia 2012

Firma wdrożyła nowy kod tradingowy bez testów. Po 45 minutach od startu giełdy stracili **440 milionów dolarów**. Firma zbankrutowała w ciągu kilku dni.

Jeden brakujący test = koniec firmy z 17-letnią historią.

Testy chronią Cię przed trzema rzeczami:

Wczesne wykrycie

Błąd na produkcji kosztuje 100× więcej niż błąd złapany przez test podczas pisania kodu.

Żywa dokumentacja

Test mówi: ta funkcja zwraca 5 dla 2 i 3. Komentarze kłamią — testy nigdy.

Bezpieczne zmiany

Refaktoryzujesz kod? Testy w 0,5 sekundy powiedzą Ci, czy niczego nie zepsułeś.

W Pythonie: unittest (wbudowany) + pytest (zewnętrzny, branżowy standard)

ideis

Testy manualne vs. automatyczne

Manualne

(klikam i sprawdzam oczami)

✓ Elastyczność i intuicja

✓ Świetne do eksploracji UX

✓ Wykrywają to, czego nie przewidziałeś

✗ Wolne — minuty na test

✗ Niepowtarzalne

✗ Zmęczenie testera = błędy

Automatyczne

(uruchamia komputer, sekundy)

✓ Szybkie — 50 testów w 0,05 s

✓ Powtarzalne 1000 razy dziennie

✓ Mierzalne pokrycie

✓ Integracja z CI/CD

✗ Wymagają napisania

✗ Nie wykryją problemów UX

Manualne ≠ złe. Automatyzujemy to, co robimy WIELE razy. Eksplorację robimy ręcznie.

ideis

Jednostkowe vs. integracyjne

Test jednostkowy

Sprawdza JEDNĄ funkcję w izolacji.

Przykład:

```
validate_email("jan@dsw.edu.pl") == True
```

Narzędzia: unittest, pytest

⚡ Szybki · niezależny · uruchamiany przy każdym commicie

Test integracyjny

Sprawdza, czy KILKA komponentów współpracuje.

Przykład:

Czy formularz rejestracji ZAPISUJE użytkownika do bazy I wysyła email powitalny?

Narzędzia: pytest, requests, selenium

🦕 Wolniejszy · wymaga bazy/API · uruchamiany rzadziej

Piramida testów branżowa ·

70% jednostkowe · **20%** integracyjne · **10%** end-to-end (UI)

Dziś skupiamy się na jednostkowych — fundamencie wszystkiego innego.

ideis

assert · najprostszy test świata

assert = TWOJE oczekiwanie. Jeśli rzeczywistość się z nim nie zgadza — program krzyczy.

Wszystko OK:

```
>>> assert 2 + 2 == 4
>>>
(nic się nie dzieje – czyli sukces)
```

Coś poszło nie tak:

```
>>> assert 2 + 2 == 5
Traceback (most recent call last):
AssertionError
```

Z opcjonalną wiadomością (warto!):

```
assert warunek, "opis błędu, gdy warunek false"

assert response.status_code == 200, "API powinno zwracać 200"
```

assert to fundament. Wszystkie biblioteki testowe (unittest, pytest) to tylko ładniejsza nakładka.

ideis

unittest

`unittest` — wbudowany w Pythona framework do pisania testów. Wzorowany na **JUnit** z Javy.

Cztery rzeczy, które musisz wiedzieć:

1. **Klasa testowa dziedziczy po unittest.TestCase**

   Testy organizujesz w klasy, nie w luźne funkcje.

2. **Metody testowe MUSZĄ zaczynać się od test_**

   Inaczej framework ich nie wykryje. To NIE jest opcja.

3. **Asercje przez self.assertEqual, self.assertTrue, ...**

   Nie używaj zwykłego assert — komunikaty błędów są gorsze.

4. **Wbudowany — żadnej instalacji**

   import unittest i jedziesz. Działa wszędzie, gdzie jest Python.

Asercje unittest · najpopularniejsze

Metoda	sprawdza
`assertEqual(a, b)`	`a == b`
`assertNotEqual(a, b)`	`a != b`
`assertTrue(x)`	`bool(x) is True`
`assertFalse(x)`	`bool(x) is False`
`assertIsNone(x)`	`x is None`
`assertIn(a, b)`	`a in b`
`assertRaises(Exception)`	`kod rzuca wyjątek`
`assertIsInstance(a, b)`	`isinstance(a, b)`

Decyzja w 3 sekundach:

Sprawdzasz wartość?

→ **`assertEqual`**

Sprawdzasz warunek bool?

→ **`assertTrue / False`**

Sprawdzasz, że wybucha?

→ **`assertRaises`**

90% testów to te trzy. Resztę zerkniesz w dokumentację, jak będzie potrzeba.

ideis

Klasa testowa · Calculator

Klasa Calculator (kod produkcyjny)

```python
class Calculator:
    def add(self, a, b):
        return a + b

    def subtract(self, a, b):
        return a - b

    def divide(self, a, b):
        if b == 0:
            raise ValueError("Dzielenie przez zero")
        return a / b
```

Klasa testowa (testy jednostkowe)

```python
import unittest
from calculator import Calculator

class TestCalculator(unittest.TestCase):
    def setUp(self):                    # Arrange
        self.calc = Calculator()

    def test_add(self):
        result = self.calc.add(1, 2)    # Act
        self.assertEqual(result, 3)         # Assert

    def test_divide_by_zero(self):
        with self.assertRaises(ValueError):
            self.calc.divide(10, 0)
```

Wzorzec AAA: **Arrange** (przygotuj dane, np. setUp) → **Act** (uruchom testowany kod) → **Assert** (sprawdź wynik). To globalny standard — zobaczysz go w każdym języku.

setUp i tearDown · izolacja testów

setUp · PRZED każdym testem

- Tworzy świeże obiekty (np. nowy Calculator)
- Każdy test startuje od czystego stanu
- Eliminuje powielanie kodu (DRY)

tearDown · PO każdym teście

- Zwalnia zasoby (pliki, połączenia DB)
- Przywraca stan do punktu zerowego
- Często niepotrzebny — i to OK

Cykl życia testu w klasie testowej:

`setUp()` → `test_add()` → `tearDown()` → `setUp()` → `test_subtract()` → `tearDown()`

Pytanie: co się stanie, jeśli zrobisz self.calc = Calculator() raz, w __init__? → brak izolacji.

ideis

DEMO · unittest · wszystko działa

Komenda:

```
$ python -m unittest test_calculator_unittest -v
```

Output:

```
test_add_mixed_numbers      (test_calculator_unittest.TestCalculator) ... ok
test_add_negative_numbers   (test_calculator_unittest.TestCalculator) ... ok
test_add_positive_numbers   (test_calculator_unittest.TestCalculator) ... ok
test_divide                 (test_calculator_unittest.TestCalculator) ... ok
test_divide_by_zero_raises_error (test_calculator_unittest.TestCalculator) ... ok
test_multiply               (test_calculator_unittest.TestCalculator) ... ok
test_subtract               (test_calculator_unittest.TestCalculator) ... ok

----------------------------------------------------------------------
Ran 10 tests in 0.001s

OK
```

Flaga -v (verbose) pokazuje nazwy testów. Zielony OK = wszystko zadziałało.

ideis

DEMO · unittest · test PADA

Co jeśli ktoś psuje kod? Zmieniamy add() żeby zwracało a + b + 1...

```
test_add_negative_numbers ... FAIL
test_add_positive_numbers ... FAIL
test_subtract             ... ok

======================================================================
FAIL: test_add_positive_numbers (test_calculator_unittest.TestCalculator)
----------------------------------------------------------------------
Traceback (most recent call last):
  File "test_calculator_unittest.py", line 23, in test_add_positive_numbers
    self.assertEqual(result, 5)
AssertionError: 6 != 5

----------------------------------------------------------------------
Ran 10 tests in 0.002s

FAILED (failures=2)
```

Jak czytać błąd: patrz na ostatnią linię → `AssertionError: 6 != 5` → test oczekiwał 5, dostał 6. Idź do test_add i sprawdź dlaczego.

Struktura projektu testowego

Testy trzymamy w osobnym katalogu — żeby przy deployu nie wrzucać ich na produkcję.

```
moj_projekt/
├── moj_modul/
│   ├── __init__.py
│   ├── calculator.py
│   └── product.py
├── tests/
│   ├── __init__.py
│   ├── test_calculator.py
│   └── test_product.py
└── requirements.txt
```

Uruchomienie wszystkich testów w katalogu (pliki muszą zaczynać się od test_):

```
$ python -m unittest discover tests
```

ideis

Najczęstsze błędy · unikaj na zadaniach

Brak prefiksu test_

Objaw: `collected 0 items`

Fix: Nazwa metody / pliku MUSI zaczynać się od test_

Zapomniane self.calc

Objaw: `AttributeError: ... has no attribute 'calc'`

Fix: W setUp przypisuj do self.calc, nie do lokalnego calc

ImportError

Objaw: `No module named 'calculator'`

Fix: Uruchamiaj testy z katalogu lab_03 (cd lab_03/)

Test zależy od kolejności

Objaw: `Test działa sam, pada w grupie`

Fix: Każdy test musi być niezależny — używaj setUp

Wracaj do tego slajdu na zadaniach — zaoszczędzisz 30 minut frustracji.

ideis

pytest · branżowy standard

`pytest` — zewnętrzny framework, który podbił branżę. Tego używają firmy. Tego się oczekuje na rozmowach.

Co dostajesz (poza unittest):

Zwykłe funkcje

Bez klas, bez self. Po prostu def test_xxx():

Goły assert

assert a == b zamiast self.assertEqual(a, b)

Lepsze błędy

Pokazuje wartości po obu stronach == automatycznie

Fixtures

Elastyczna alternatywa dla setUp z DI

Parametryzacja

Jeden test, wiele zestawów danych

Wtyczki

pytest-cov, pytest-django, pytest-xdist...

Instalacja: pip install pytest

ideis

unittest vs pytest

	unittest	pytest
Struktura	`Klasy + metody`	`Zwykłe funkcje`
Asercje	`self.assertEqual(a, b)`	`assert a == b`
Przygotowanie	`setUp / tearDown`	`@pytest.fixture`
Instalacja	`Wbudowany`	`pip install pytest`
Komunikaty	`Ogólne`	`Szczegółowe (diff)`

Kiedy co?

- Nowy projekt? → **`pytest`**
- Stary projekt z unittest? → **`zostań przy unittest`**

Branża dziś: ~85% nowych projektów Python = pytest. *Gdziekolwiek pójdziesz pracować, prawdopodobnie spotkasz pytest.*

Fixtures · pytestowy odpowiednik setUp

```python
import pytest
from calculator import Calculator

@pytest.fixture
def calc():                          # definicja fixture
    return Calculator()

def test_add(calc):        # argument o tej samej nazwie!
    assert calc.add(2, 3) == 5

def test_multiply(calc):  # ten sam fixture, świeży obiekt
    assert calc.multiply(3, 4) == 12
```

Jak to działa?

Pytest patrzy na **nazwy argumentów** funkcji testowej i szuka fixture o tej samej nazwie. Sam je tworzy i przekazuje.

Ten wzorzec to:

Dependency Injection

To samo używa Spring (Java), Angular, FastAPI, NestJS. Termin, który usłyszysz 100 razy w karierze.

Fixture = setUp z unittest. Ten sam koncept, prostsza składnia, bardziej elastyczny.

ideis

Parametryzacja · jeden test, wiele danych

```python
import pytest

@pytest.mark.parametrize("a, b, expected", [
    (2, 3, 5),
    (-1, 1, 0),
    (0, 0, 0),
    (100, -50, 50),
])
def test_add(a, b, expected):
    calc = Calculator()
    assert calc.add(a, b) == expected
```

...a w outputcie:

```
test_add[2-3-5] PASSED
test_add[-1-1-0] PASSED
test_add[0-0-0] PASSED
test_add[100--50-50] PASSED

4 passed in 0.02s
```

Każdy zestaw ma swój ID — gdy padnie, dokładnie wiesz który.

Jeden test, ale pytest uruchomi go 4 razy — osobno dla każdego zestawu!

ideis

Marks · oznaczanie testów

marks pozwalają oznaczać testy, żeby je selektywnie uruchamiać, pomijać lub oczekiwać porażki.

@pytest.mark.skip — pomija test bezwarunkowo

```
@pytest.mark.skip(reason="Funkcja jeszcze nie zaimplementowana")
```

@pytest.mark.xfail — test ma prawo nie przejść (np. znany bug)

```
@pytest.mark.xfail(reason="Znany bug #123")
```

Własne marks — selektywne uruchamianie:

```
$ pytest -m smoke      # uruchomi tylko testy oznaczone @pytest.mark.smoke
```

W skróconym outpucie litery oznaczają:

. passed · **F** failed · **s** skipped · **x** xfail · **E** error (błąd w samym tescie)

DEMO · pytest · pełny output

```
$ pytest test_calculator_pytest.py -v
```

```
test_calculator_pytest.py::test_add PASSED                          [  9%]
test_calculator_pytest.py::test_subtract PASSED                     [ 18%]
test_calculator_pytest.py::test_multiply PASSED                     [ 27%]
test_calculator_pytest.py::test_divide PASSED                       [ 36%]
test_calculator_pytest.py::test_divide_by_zero PASSED               [ 45%]
test_calculator_pytest.py::test_add_parametrized[2-3-5] PASSED      [ 54%]
test_calculator_pytest.py::test_add_parametrized[-1-1-0] PASSED     [ 63%]
test_calculator_pytest.py::test_add_parametrized[0-0-0] PASSED      [ 72%]
test_calculator_pytest.py::test_skipped_example SKIPPED             [ 81%]
test_calculator_pytest.py::test_xfail_example XFAIL                  [100%]

============= 8 passed, 1 skipped, 1 xfailed in 0.04s =============
```

Zauważ: parametrized pokazuje [2-3-5] [-1-1-0] [0-0-0] — każdy zestaw osobno.

ideis

Czego dziś NIE zdążyliśmy

To dopiero początek — w prawdziwym projekcie spotkasz jeszcze:

Mockowanie

`unittest.mock`

Gdy kod używa API, bazy, plików — symulujesz je, żeby test był szybki i niezależny.

Pokrycie kodu (coverage)

`pytest-cov`

pytest --cov mówi, ile linii kodu jest testowanych. Cel branżowy: 80%+.

TDD

`metodyka`

Test-Driven Development: najpierw piszesz test, potem kod, który ten test spełnia.

CI/CD

`DevOps`

Testy odpalają się automatycznie po każdym push do repozytorium (GitHub Actions, GitLab CI).

Selenium / Playwright

`E2E`

Testowanie aplikacji webowych w prawdziwej przeglądarce — symulacja klikania użytkownika.

Property-based testing

`hypothesis`

Hypothesis losuje setki przypadków testowych za Ciebie. Łapie błędy, których byś nie wymyślił.

Zapamiętaj te hasła. Wygoogluj je w domu — to mapa Twojej dalszej nauki.

ideis

Zadania

Zadanie 1 · unittest

Stwórz klasę **`Product`** (pola: name, price, quantity) z metodami: `add_stock()`, `remove_stock()`, `is_available()`, `total_value()`.

Napisz testy **unittest** z użyciem `setUp` i `assertRaises` (np. cena ujemna → ValueError).

💡 ***Tip:*** *zacznij od najprostszego testu —* ***`test_is_available_returns_false_for_empty_product`****. Potem stopniowo dodawaj kolejne.*

Zadanie 2 · pytest

Przepisz testy z Zadania 1 używając pytest:

- Użyj `@pytest.fixture` do inicjalizacji obiektu Product
- Użyj `@pytest.mark.parametrize` do testowania `add_stock` z różnymi danymi
- Użyj `with pytest.raises(ValueError)` do testowania błędów

Zapisz: product.py · test_product_unittest.py · test_product_pytest.py · Czas: ~20 min

Utknąłeś? Zerknij na slajd „Najczęstsze błędy” — odpowiedź pewnie tam jest.

Sprawdź, co umiesz

Pomyśl 30 sekund nad każdym pytaniem. Nie odpowiadaj na głos — to ma być Twoja głowa, nie cudza.

1. Co robi setUp i dlaczego jest ważny?
2. Czym różni się assertEqual od assertRaises?
3. Po co istnieje @pytest.fixture? Co zastępuje?
4. Co znaczy litera s w skróconym outpucie pytest?
5. Jeden test, 5 zestawów danych — czego użyjesz?
6. Test pada z AssertionError: 6 != 5. Co to mówi?

Nie umiesz na któreś? To nie wstyd — wróć do odpowiedniego slajdu PRZED następnymi zajęciami.

ideis

Dziękuję za uwagę

PYTANIA?